

Carnegie Mellon University

Real-Time Posture Correction and Kinematic Alignment

Jenny Choung, Ryan Lee, Cindy Woo

Problem, Plan, and Preliminary Results



Evaluate and correct yoga form in real-time using a standard webcam



Tools:



MediaPipe

Google's MediaPipe library



extract precise joint coordinates as the user moves



Jacobi SVD Method



solve the alignment problem, computing the optimal rotation needed to map the user's body data to instructor-validated ground truth poses



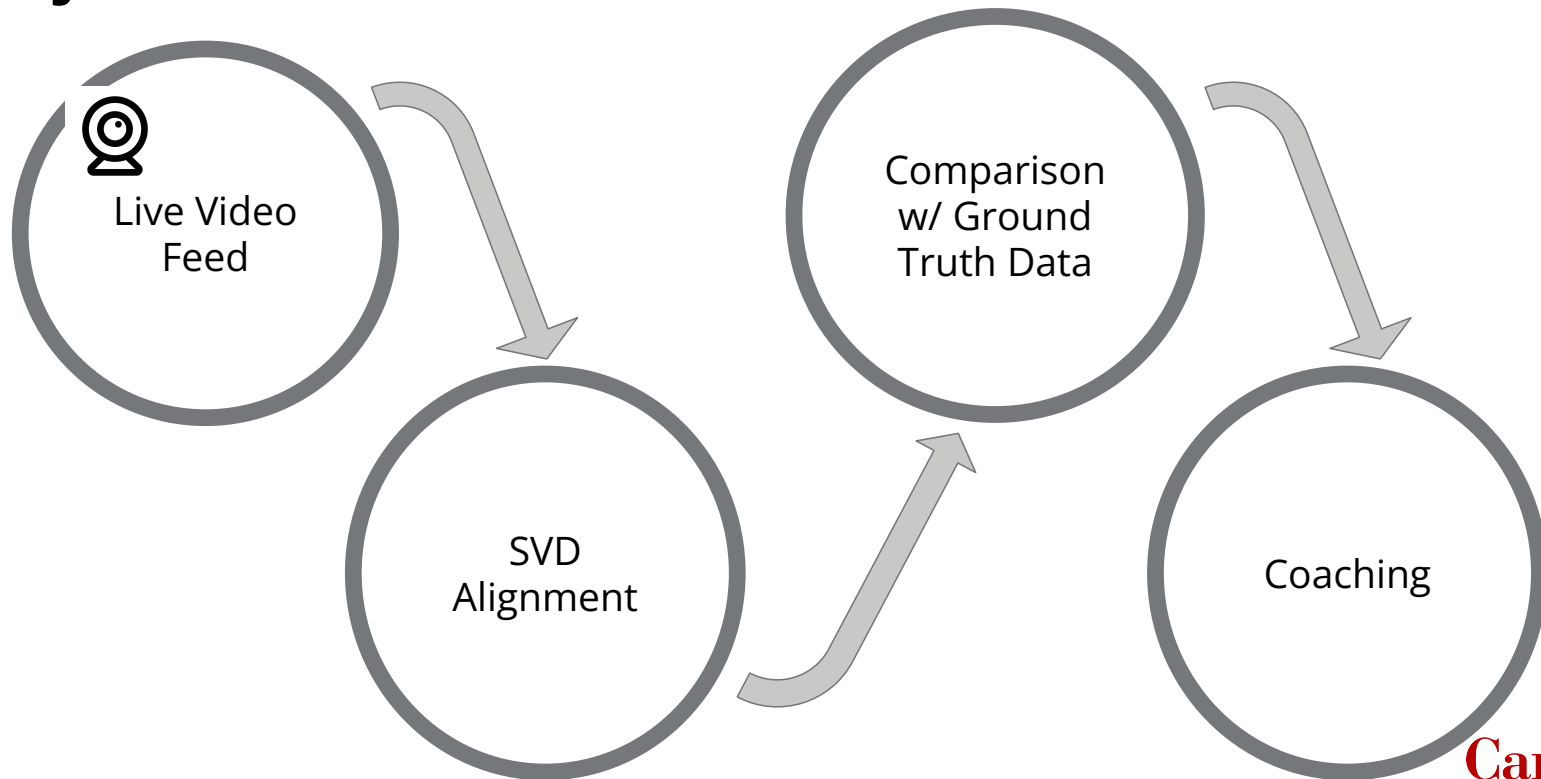
Output a performance score 1-5



Real-time digital coach through specific, easy-to-follow advice for correcting the posture



Project Overview



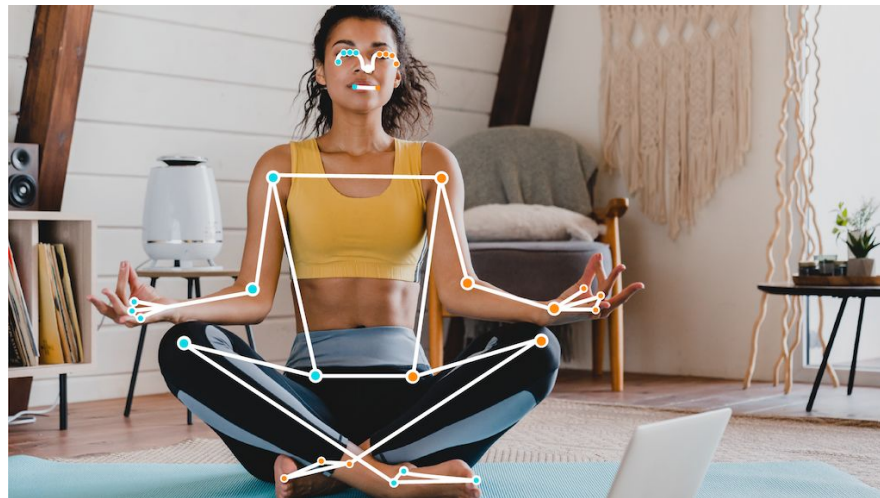
Google MediaPipe: Pose Landmarker

Google MediaPipe

- Open-source framework from Google
- Used for building cross-platform machine learning (ML) pipelines
- Provides up to 33 joint coordinates

MediaPipe Pose Landmarker

- Detect landmarks of human bodies in an image or video
- Use to identify key body locations, analyze posture, and categorize movements
- Output body pose landmarks in image coordinates and in 3-dimensional world coordinates.



Source: https://ai.google.dev/edge/mediapipe/solutions/vision/pose_landmarker

Building Ground Truth Data

"Balancing Table" Pose



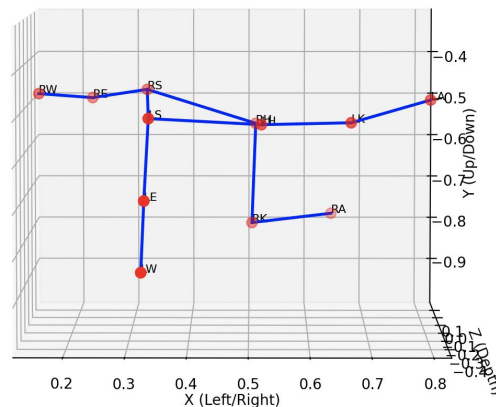
3D Joint Coordinates and **Joint Angles** are obtained for 5 instructor images for every pose and averaged

Averaged Biomechanical Angles

Left Elbow: 160.28°
Right Elbow: 114.07°
Left Shoulder: 104.87°
Right Shoulder: 128.3°
Left Hip: 124.5°
Right Hip: 102.81°
Left Knee: 140.2°
Right Knee: 74.85°

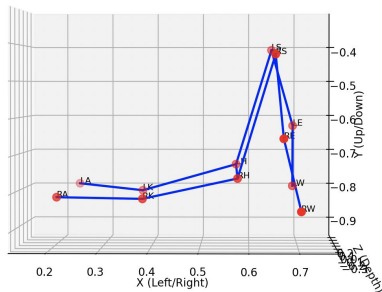
Ground Truth:

- 12 joint coordinates
- 8 joint angles

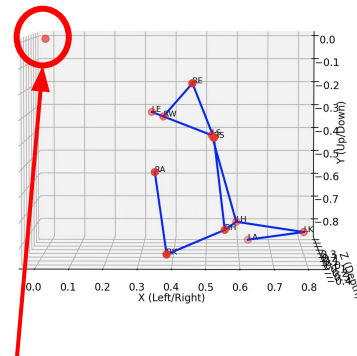


Ground Truth Data Collection for Side-Facing Poses

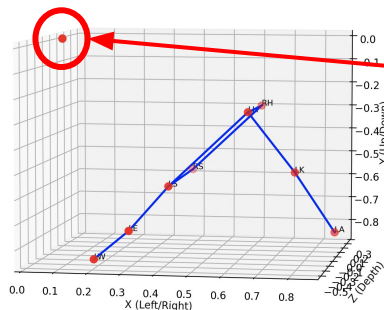
"Cobra" Pose



"One-legged King Pigeon" Pose



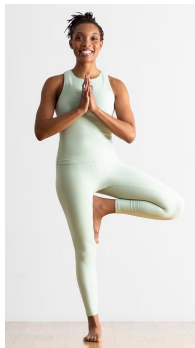
"Downward Facing Dog" Pose



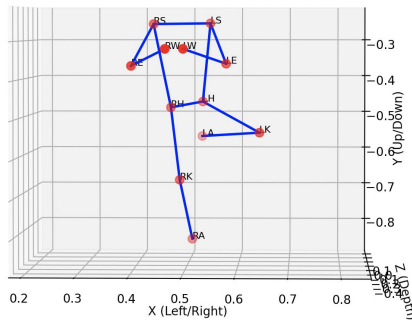
- Disregarded low-confidence "hidden" joint using a dynamic visibility threshold (*logged in the matrix as $[0,0,0]$*)
- Compared only mutually visible joints within the SVD algorithm (User vs. GT)

Z-Depth Ambiguity Problem for Front-Facing Poses

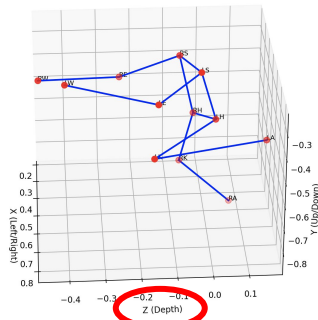
"Tree" Pose



Front View



Side View

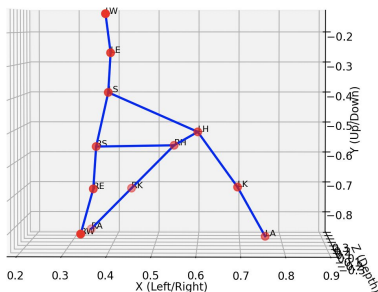


Single-lens cameras cannot accurately resolve longitudinal depth from front-facing perspectives

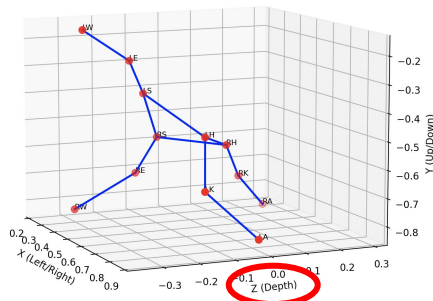
"Triangle" Pose



Front View

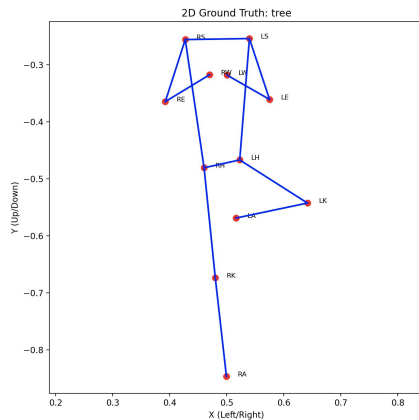
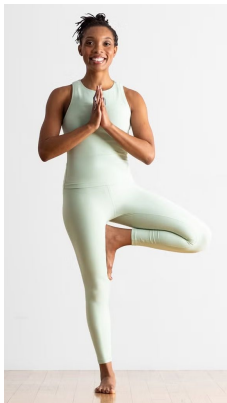


From an angle

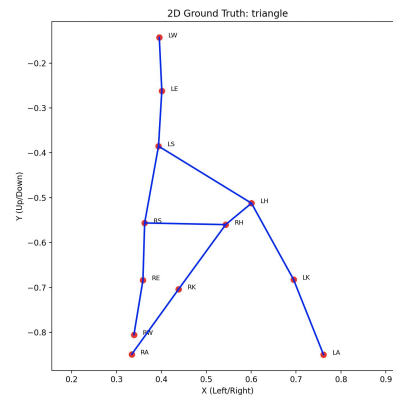


Solution for Z-Depth Ambiguity

“Tree” Pose



“Triangle” Pose



Hybrid Dimensionality Model

- Evaluating side-facing poses in 3D and forward-facing poses in 2D
- Flattening coordinate matrices to 2D eliminates unreliable Z-axis data

Initial Pose Alignment

Matrix pre-processing

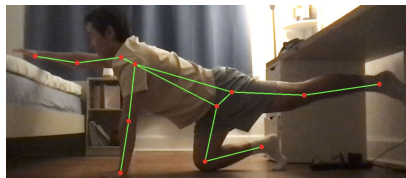
$$M_{\text{norm}} = (M - \text{mean}(M)) / \|M - \text{mean}(M)\|_F$$

“M” is a joint coordinate matrix of shape $m \times n$:

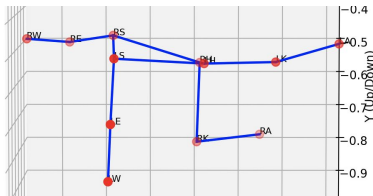
- **m**: # of valid joints (max = 12)
- **n**: # of axis (3D $\rightarrow$ x, y, z)

Frobenius norm

Matrix A (User)



Matrix B (Ground Truth)



1. Centralization

2. Normalization

Next up:

Rotation!

Optimal Rotation Extraction with Jacobi SVD Method

Jacobi SVD Pipeline:

1. Form Symmetry Matrix

$$S = C^T \cdot C$$

2. Core Jacobi loop

$$\text{max_off_diagonal} \rightarrow \text{theta} \rightarrow G$$

3. Apply rotation and accumulate it to V

$$S = G^T \cdot S \cdot G \text{ and } V = V \cdot G$$

4. Extract singular value

$$\text{sigma} = \text{np.sqrt}(\text{eigenvalues})$$

5. Recover U

$$U[:,i] = \text{np.dot}(C, V[:,i]) / \text{sigma}[i]$$

```
def jacobi_svd(C, tolerance=1e-9, max_iter=100):
```

```
    n = C.shape[0]
```

```
    S = C.T @ C
```

```
    V = np.eye(n)
```

$$C \text{ (covariance matrix)} = A^T \cdot B$$

```
    for iter in range(max_iter):
        max_off_diag, p, q = max_off_diagonal_numpy(S)
        if max_off_diag < tolerance:
            break
        if abs(S[p, p] - S[q, q]) < 1e-15):
            theta = np.pi / 4.0
        else:
            theta = 0.5 * np.arctan(2 * S[p, q] / (S[p, p] - S[q, q]))
        G = np.eye(n)
        G[p, p] = np.cos(theta)
        G[q, q] = np.cos(theta)
        G[p, q] = -np.sin(theta)
        G[q, p] = np.sin(theta)
        S = G.T @ S @ G
        V = V @ G
```

repeat until "S" is nearly diagonal

```
    eigenvalues = np.diag(S)
    eigenvalues = np.maximum(eigenvalues, 0.0)
    sigma = np.sqrt(eigenvalues)
```

Forming the Given's Matrix G

```
    U = np.zeros((n, n))
    for i in range(n):
        if sigma[i] > tolerance:
            U[:, i] = np.dot(C, V[:, i]) / sigma[i]
        else:
            U[i, i] = 1.0
    return U, sigma, V.T
```

$$\text{Finally, } R = U \cdot V^T$$

Optimal Rotation Extraction with Jacobi SVD Method

Jacobi SVD Pipeline:

1. Form Symmetry Matrix

$$\mathbf{S} = \mathbf{C}^T \cdot \mathbf{C}$$

2. Core Jacobi loop

$$\text{max_off_diagonal} \rightarrow \text{theta} \rightarrow \mathbf{G}$$

3. Apply rotation and accumulate it to V

$$\mathbf{S} = \mathbf{G}^T \cdot \mathbf{S} \cdot \mathbf{G} \text{ and } \mathbf{V} = \mathbf{V} \cdot \mathbf{G}$$

4. Extract singular value when done

$$\Sigma = \text{np.sqrt}(\text{eigenvalues})$$

5. Recover U

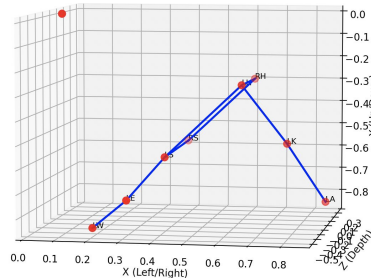
$$\mathbf{U}[:,i] = \text{np.dot}(\mathbf{C}, \mathbf{V}[:,i]) / \text{sigma}[i]$$

SVD Check against built in NumPy “linalg.svd”

Example: Downward Facing Dog Pose



Test Image



Ground Truth (N=5)

Error outputs for both SVDs after applying R

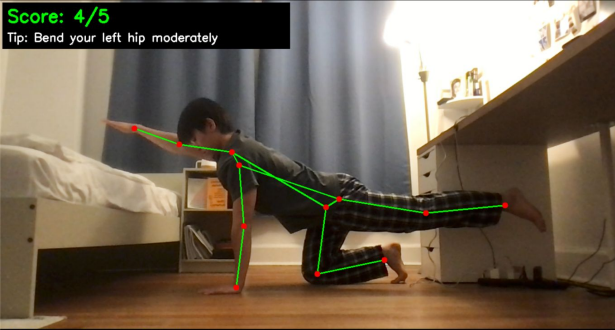
```
=====
Validating: Downward Facing Dog Pose

Image: downward_facing_dog_test.jpg
2D Alignment Error -> NumPy: 0.1129 | Jacobi: 0.1129
3D Alignment Error -> NumPy: 0.1047 | Jacobi: 0.1047
=====
```

Exact Same!

Score: 4/5

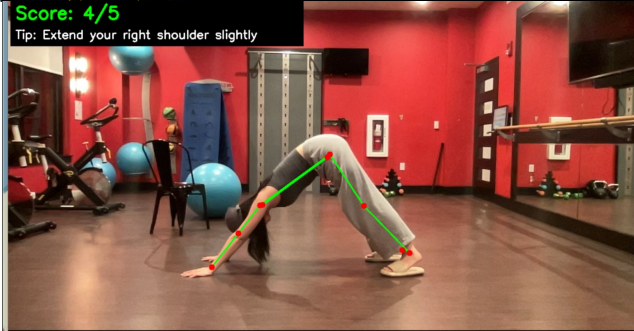
Tip: Bend your left hip moderately



SPACE: Retry | M: Menu | Q: Quit

Score: 4/5

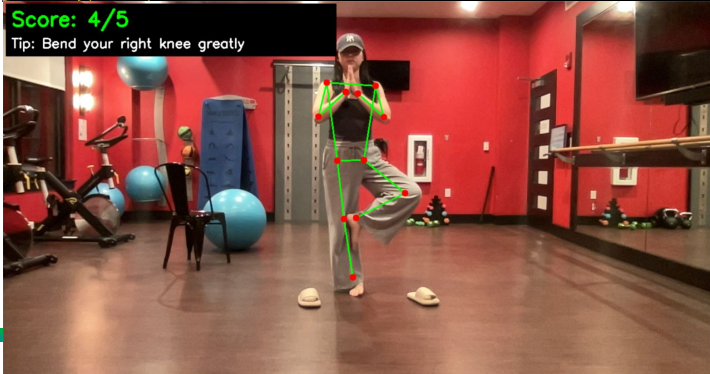
Tip: Extend your right shoulder slightly



SPACE: Retry | M: Menu | Q: Quit

Score: 4/5

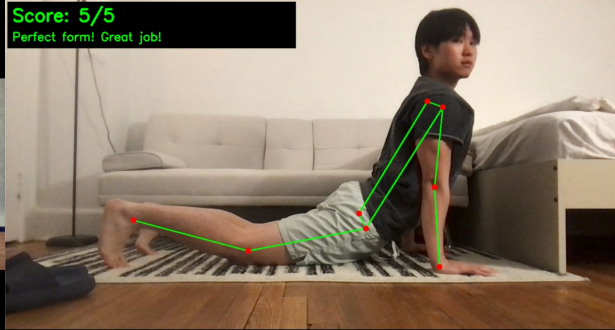
Tip: Bend your right knee greatly



SPACE: Retry | M: Menu | Q: Quit

Score: 5/5

Perfect form! Great job!



SPACE: Retry | M: Menu | Q: Quit

Score: 4/5

Tip: Extend your right hip slightly



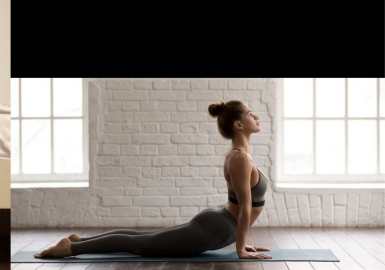
SPACE: Retry | M: Menu | Q: Quit

Score: 3/5

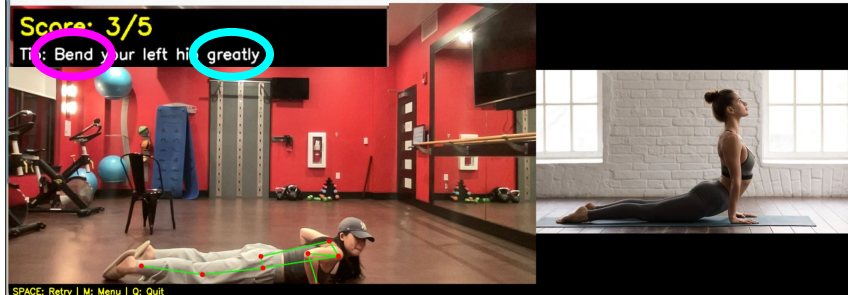
Tip: Extend your left shoulder slightly



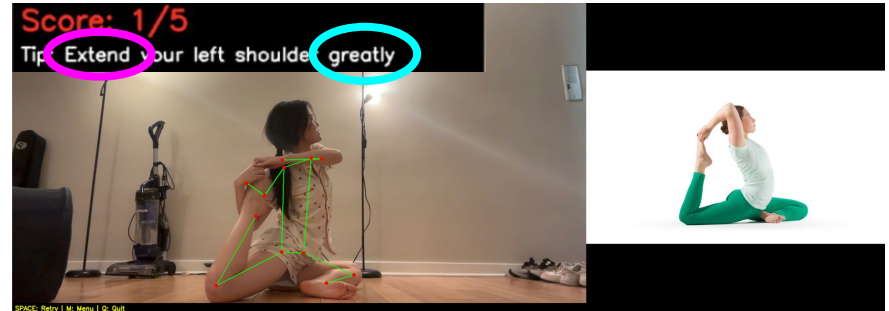
SPACE: Retry | M: Menu | Q: Quit



Feedback Mechanism



- Feedback is provided for the specific joint that exhibits the greatest joint angle difference compared to ground truth
- “Extend” or “Bend” based on sign of difference
- Adverbs, including “Slightly, Moderately, Greatly”, based on its magnitude



Result

✓ System Performance

- Real-time posture correction using webcam input
- Stable pose detection via 5s dwell time threshold
- Outputs score (1–5) + joint-specific feedback



Quantitative Results

- Evaluated across 6 yoga poses
- Uses 12 joints + 8 joint angles
- Accurate alignment via SVD-based rotation

• Handles:

- Mirror matching (left/right users)
- Missing joints via visibility threshold

• Robust to:

- Scale and position (via normalization)

Thank you!

Any questions?